

Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики и радиоэлектроники»

Кафедра программного обеспечения информационных технологий

ОТЧЁТ
по лабораторной работе №2
по дисциплине «Методы машинного обучения»
Знакомство с инструментами сбора и подготовки данных

Выполнил: студент гр. 351002
Емельяненко Е.Д.

Проверил: Лебедев А.В.

СОДЕРЖАНИЕ

1	Цель выполнения лабораторной работы	3
2	Теория	3
2.1	Разведочный анализ данных	3
2.2	Пропущенные значения	3
2.3	Выбросы и аномальные значения	4
2.4	Генерация признаков	4
2.5	Кодирование категориальных признаков	4
2.6	Стандартизация и нормализация	5
3	Формулировка индивидуального задания	5
4	Код решения индивидуального задания	6
4.1	Импорт библиотек и данных	6
4.2	Проверка на дубликаты	6
4.3	Анализ и заполнение пропусков	6
4.4	Анализ и устранение выбросов	7
4.5	Генерация новых признаков	8
4.6	Кодирование категориальных признаков	8
4.7	Стандартизация и нормализация	8
5	Скриншоты выполнения индивидуального задания	9
6	Выводы по лабораторной работе	13

1 ЦЕЛЬ ВЫПОЛНЕНИЯ ЛАБОРАТОРНОЙ РАБОТЫ

Изучить основные инструменты сбора и подготовки данных для анализа средствами Python (Pandas, NumPy, Matplotlib, Seaborn): выполнить импорт набора данных, обнаружить и обработать пропуски и аномальные значения, сгенерировать новые признаки, закодировать категориальные признаки и выполнить стандартизацию и нормализацию данных.

2 ТЕОРИЯ

2.1 Разведочный анализ данных

Разведочный анализ данных (Exploratory Data Analysis, EDA) — этап работы с данными, на котором изучается структура набора, проверяется его качество и выявляются закономерности с помощью статистических методов и визуализации. В работе используется набор `fifa_eda`, содержащий данные об игроках симулятора FIFA 19. Первичное знакомство с набором выполняется методом `df.info()`, который показывает число записей, названия столбцов, типы данных и количество непустых значений в каждом столбце. Дополнительно проверяется отсутствие повторяющихся идентификаторов и полных дубликатов строк (`nunique()`, `duplicated()`).

2.2 Пропущенные значения

Пропуск (NaN) возникает, когда значение признака для объекта не зафиксировано. Количество пропусков определяется сочетанием `df.isna().sum()`, а их расположение удобно оценивать по тепловой карте (`sns.heatmap(df.isna())`): каждая строка таблицы отображается линией, пропуск выделяется контрастным цветом. Если пропуски в разных столбцах расположены на одной высоте, они связаны между собой (данные потеряны у одного и того же объекта). Одиночные пропуски чаще являются случайными.

Основные способы заполнения:

- константа, имеющая смысл для признака (например, значение «Free Agent» для игрока без клуба);
- медиана — устойчива к выбросам и подходит для числовых признаков с асимметричным распределением;
- мода — наиболее частое значение, подходит для дискретных и порядковых признаков (например, шкала 1–5);

- групповая медиана — медиана внутри группы объектов с одинаковым значением связанного признака (`groupby(...).transform('median')`); учитывает зависимость между признаками.

Заполнение медианой или модой сохраняет типичное значение, но уменьшает разброс признака, поэтому применяется при небольшой доле пропусков.

2.3 Выбросы и аномальные значения

Выброс — значение, сильно отличающееся от основной массы наблюдений. Для их обнаружения используются диаграмма размаха (BoxPlot), гистограммы распределения и правило межквартильного размаха (IQR). Пусть Q_1 и Q_3 — первый и третий квартили, $IQR = Q_3 - Q_1$. Значения вне интервала

$$[Q_1 - 1,5 \cdot IQR; Q_3 + 1,5 \cdot IQR]$$

считаются выбросами. Для их устранения можно удалить строки либо ограничить значения границами интервала (метод `clip`, винзоризация); во втором случае объём данных сохраняется. Признаки с сильно скошенным распределением (денежные величины) предварительно преобразуются логарифмированием $x' = \ln(1 + x)$ (функция `np.log1p`), которое уменьшает влияние крайне больших значений. После обработки проверка выполняется повторным построением BoxPlot и повторным подсчётом выбросов по правилу IQR.

2.4 Генерация признаков

Новые признаки создаются на основе имеющихся и отражают важные для задачи свойства объектов: разность признаков (запас роста игрока = `Potential - Overall`), интервал времени (стаж в клубе, число лет до конца контракта), разбиение непрерывного признака на интервалы (`pd.cut`), отношение величин.

2.5 Кодирование категориальных признаков

Модели работают с числами, поэтому категориальные признаки кодируются:

- бинаризация — преобразование признака в 0/1 по условию (например, правша/левша, возраст не более 21 года);
- one-hot encoding (OHE) — для каждой категории создаётся отдельный бинарный столбец (`pd.get_dummies`);

- укрупнение категорий — редкие категории или большое число значений (позиции игроков, национальность) объединяются в небольшое число групп, чтобы избежать взрывного роста числа столбцов.

2.6 Стандартизация и нормализация

Признаки с разными единицами измерения и диапазонами приводят к единому масштабу. Стандартизация (z-преобразование) приводит признак к нулевому среднему и единичному стандартному отклонению:

$$z = \frac{x - \mu}{\sigma}.$$

Нормализация (Min-Max) приводит значения к диапазону $[0; 1]$:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}.$$

Масштабируются только непрерывные и порядковые признаки; бинарные и ОНЕ-столбцы уже лежат в диапазоне $[0; 1]$. Оба преобразования линейны и не меняют форму распределения.

3 ФОРМУЛИРОВКА ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

1. Импортировать датасет (исходные данные можно выбрать самостоятельно на [kaggle.com](https://www.kaggle.com) либо взять один из файлов, прикреплённых к заданию).
2. Проверить данные на пропуски и наличие аномальных значений, используя статистические методы и средства графической визуализации (графики распределения, диаграммы, графики типа BoxPlot, тепловые карты и т. п.). Заполнить пропуски и устранить выбросы. Проверить с помощью графических средств, что пропуски и аномальные значения отсутствуют.
3. Сгенерировать новые признаки (не менее двух), а также применить кодирование категориальных признаков (ОНЕ-кодирование, бинаризацию признаков и др.).
4. Выполнить стандартизацию и/или нормализацию данных.

4 КОД РЕШЕНИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

4.1 Импорт библиотек и данных

```
# Импорт библиотек
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

fifa_df = pd.read_csv('fifa.csv')
fifa_df.info()
```

4.2 Проверка на дубликаты

```
print("Уникальных ID:", fifa_df["ID"].nunique())
print("Повторяющихся ID:", fifa_df["ID"].duplicated().sum())

print("Полных дубликатов строк:", fifa_df.duplicated().sum())
```

4.3 Анализ и заполнение пропусков

```
na = fifa_df.isna().sum()
na = na[na > 0].to_frame('Пропусков')
na['Доля, %'] = (na['Пропусков'] / len(fifa_df) * 100).round(2)
display(na)

plt.figure(figsize=(12, 5))
sns.heatmap(fifa_df.isna(), cbar=False, yticklabels=False, cmap='viridis')
plt.title('Карта пропусков (жёлтое = пропуск)')
plt.show()
```

```
df = fifa_df.copy()

# Club: пропуск = игрок без клуба
df['Club'] = df['Club'].fillna('Free Agent')

# Contract Valid Until: оставляем только год, затем заполняем медианой
df['Contract Valid Until'] = pd.to_numeric(
    df['Contract Valid Until'].astype(str).str.extract(r'(\d{4})')[0])
df['Contract Valid Until'] = df['Contract Valid Until'].fillna(df['Contract Valid Until'].median())

# Value: медиана среди игроков с тем же Overall (стоимость зависит от рейтинга)
df['Value'] = df['Value'].fillna(df.groupby('Overall')['Value'].transform('median'))
df['Value'] = df['Value'].fillna(df['Value'].median()) # если для какого-то Overall медианы нет

# Дискретные признаки 1-5: заполняем модой
for col in ['International Reputation', 'Skill Moves']:
    df[col] = df[col].fillna(df[col].mode()[0])

df.info()
```

```
print('Всего пропусков:', df.isna().sum().sum())
```

```
plt.figure(figsize=(12, 5))
sns.heatmap(df.isna(), cbar=False, yticklabels=False, cmap='viridis')
plt.title('Карта пропусков после заполнения')
plt.show()
```

4.4 Анализ и устранение выбросов

```
num_cols = ['Age', 'Overall', 'Potential', 'Value', 'Wage',
            'Release Clause', 'Height', 'Weight']

fig, axes = plt.subplots(2, 4, figsize=(18, 8))
for ax, col in zip(axes.ravel(), num_cols):
    sns.boxplot(y=df[col], ax=ax, color='skyblue')
    ax.set_title(col)
plt.suptitle('BoxPlot до обработки выбросов')
plt.tight_layout()
plt.show()

def iqr_bounds(s):
    q1, q3 = s.quantile([0.25, 0.75])
    i = q3 - q1
    return q1 - 1.5 * i, q3 + 1.5 * i

rep = []
for col in num_cols:
    lo, hi = iqr_bounds(df[col])
    rep.append([col, lo, hi, ((df[col] < lo) | (df[col] > hi)).sum()])
display(pd.DataFrame(rep, columns=['Признак', 'Нижняя граница', 'Верхняя граница', 'Число выбросов']))
```

1. Логарифмирование сильно скошенных денежных признаков

```
money = ['Value', 'Wage', 'Release Clause']
for col in money:
    df[col] = np.log1p(df[col])
```

2. Ограничение по границам IQR

```
for col in num_cols:
    lo, hi = iqr_bounds(df[col])
    df[col] = df[col].clip(lo, hi)
```

```
fig, axes = plt.subplots(2, 4, figsize=(18, 8))
for ax, col in zip(axes.ravel(), num_cols):
    sns.boxplot(y=df[col], ax=ax, color='lightgreen')
    ax.set_title(col)
plt.suptitle('BoxPlot после обработки выбросов')
plt.tight_layout()
plt.show()

print('Выбросов по IQR после обработки:')
for col in num_cols:
    lo, hi = iqr_bounds(df[col])
    print(f'{col}: {((df[col] < lo) | (df[col] > hi)).sum()}')
```

```
df[num_cols].hist(bins=30, figsize=(16, 8))
plt.suptitle('Распределения после очистки')
plt.tight_layout()
```

```
plt.show()

plt.figure(figsize=(9, 7))
sns.heatmap(df[num_cols].corr(), annot=True, fmt='.2f', cmap='coolwarm')
plt.title('Корреляции числовых признаков')
plt.show()
```

4.5 Генерация новых признаков

Данные соответствуют FIFA 19 (2018/2019), поэтому опорный год = 2018
REF_YEAR = 2018

```
df['Potential_Gap'] = df['Potential'] - df['Overall'] # запас роста
df['Years_In_Club'] = (REF_YEAR - df['Joined']).clip(lower=0) # стаж в клубе
df['Contract_Years'] = (df['Contract Valid Until'] - REF_YEAR).clip(lower=0) # лет до конца контракта
df['Age_Group'] = pd.cut(df['Age'], bins=[0, 21, 26, 31, 100],
                        labels=['Young', 'Prime', 'Experienced', 'Veteran'])

df[['Potential_Gap', 'Years_In_Club', 'Contract_Years', 'Age_Group']].head()
```

4.6 Кодирование категориальных признаков

Бинаризация признаков

```
df['Is_Free_Agent'] = (df['Club'] == 'Free Agent').astype(int)
df['Is_Right_Foot'] = (df['Preferred Foot'] == 'Right').astype(int)
df['Is_Young'] = (df['Age'] <= 21).astype(int)
df['Is_Star'] = (df['Overall'] >= df['Overall'].quantile(0.9)).astype(int)
```

```
pos_map = {
    'GK': 'GK',
    'CB': 'DEF', 'LCB': 'DEF', 'RCB': 'DEF', 'LB': 'DEF', 'RB': 'DEF', 'LWB': 'DEF', 'RWB': 'DEF',
    'CM': 'MID', 'LCM': 'MID', 'RCM': 'MID', 'CDM': 'MID', 'LDM': 'MID', 'RDM': 'MID',
    'CAM': 'MID', 'LAM': 'MID', 'RAM': 'MID', 'LM': 'MID', 'RM': 'MID',
    'ST': 'FWD', 'LS': 'FWD', 'RS': 'FWD', 'CF': 'FWD', 'LF': 'FWD', 'RF': 'FWD',
    'LW': 'FWD', 'RW': 'FWD'
}
df['Position_Group'] = df['Position'].map(pos_map).fillna('Other')
```

Nationality имеет много категорий: оставляем топ-10, остальные объединяем в Other

```
top_nat = df['Nationality'].value_counts().head(10).index
df['Nationality_Top'] = df['Nationality'].where(df['Nationality'].isin(top_nat), 'Other')
```

```
df = pd.get_dummies(df, columns=['Position_Group', 'Age_Group', 'Nationality_Top'],
                    prefix=['Pos', 'AgeG', 'Nat'], dtype=int)
```

Служебные и высокоуникальные столбцы больше не нужны

```
df = df.drop(columns=['ID', 'Name', 'Nationality', 'Club', 'Position', 'Preferred Foot'])
print(df.shape)
df.head()
```

4.7 Стандартизация и нормализация

```
scale_cols = ['Age', 'Overall', 'Potential', 'Value', 'Wage', 'Release Clause',
```

```

'Height', 'Weight', 'International Reputation', 'Skill Moves',
'Potential_Gap', 'Years_In_Club', 'Contract_Years']

# Стандартизация
df_std = df.copy()
df_std[scale_cols] = (df[scale_cols] - df[scale_cols].mean()) / df[scale_cols].std()

# Нормализация
df_mm = df.copy()
rng = df[scale_cols].max() - df[scale_cols].min()
df_mm[scale_cols] = (df[scale_cols] - df[scale_cols].min()) / rng.replace(0, 1)

print('Стандартизация: mean ≈ 0, std ≈ 1')
display(df_std[scale_cols].describe().loc[['mean', 'std']].round(3))
print('Нормализация: min = 0, max = 1')
display(df_mm[scale_cols].describe().loc[['min', 'max']].round(3))

print(df_std[scale_cols].isna().sum().sum(), np.isinf(df_std[scale_cols]).sum().sum())
print(df_mm[scale_cols].isna().sum().sum(), np.isinf(df_mm[scale_cols]).sum().sum())

fig, axes = plt.subplots(1, 3, figsize=(18, 4))
sns.histplot(df['Overall'], kde=True, ax=axes[0]).set_title('Overall: исходный')
sns.histplot(df_std['Overall'], kde=True, ax=axes[1], color='orange').set_title('Overall: стандартизация')
sns.histplot(df_mm['Overall'], kde=True, ax=axes[2], color='green').set_title('Overall: Min-Max')
plt.tight_layout()
plt.show()

```

5 СКРИНШОТЫ ВЫПОЛНЕНИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

С помощью функции `df.info()` ознакомимся с общей информацией о датасете:

- количеством записей;
- столбцами датафрейма;
- типами данных каждого столбца;
- количеством ненулевых значений в каждом столбце.

```

[3]: fifa_df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 18207 entries, 0 to 18206
Data columns (total 18 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   ID                                         18207 non-null  int64
1   Name                                       18207 non-null  object
2   Age                                        18207 non-null  int64
3   Nationality                             18207 non-null  object
4   Overall                                   18207 non-null  int64
5   Potential                                18207 non-null  int64
6   Club                                      17966 non-null  object
7   Value                                    17955 non-null  float64
8   Wage                                      18207 non-null  float64
9   Preferred Foot                            18207 non-null  object
10  International Reputation                 18159 non-null  float64
11  Skill Moves                             18159 non-null  float64
12  Position                                 18207 non-null  object
13  Joined                                   18207 non-null  int64
14  Contract Valid Until                     17918 non-null  object
15  Height                                   18207 non-null  float64
16  Weight                                   18207 non-null  float64
17  Release Clause                           18207 non-null  float64
dtypes: float64(7), int64(5), object(6)
memory usage: 2.5+ MB

```

Рис. 1 – Общая информация о датасете (`df.info()`)

	Пропусков	Доля, %
Club	241	1.32
Value	252	1.38
International Reputation	48	0.26
Skill Moves	48	0.26
Contract Valid Until	289	1.59

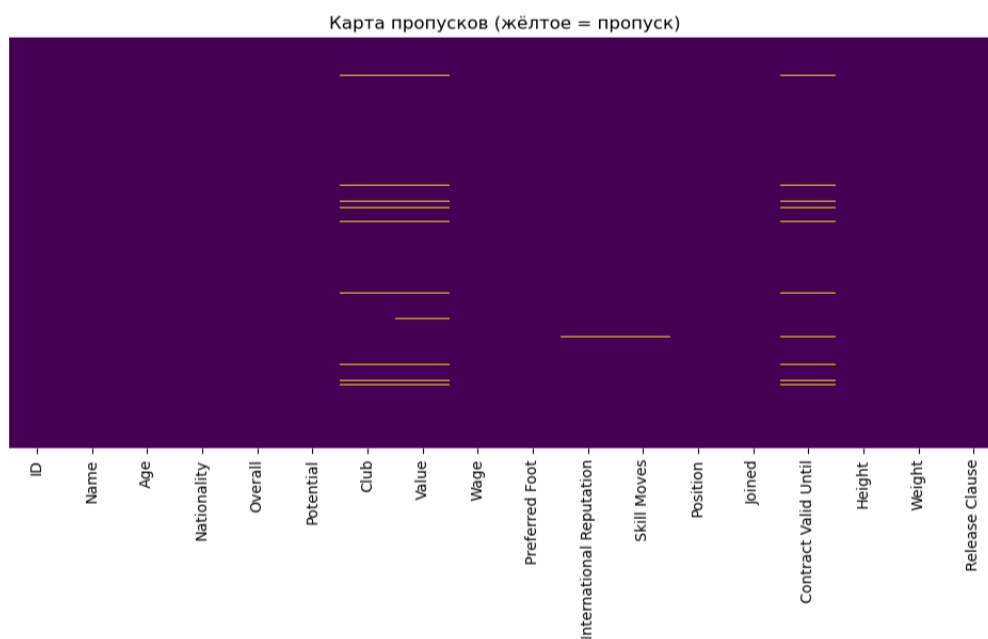


Рис. 2 – Карта пропусков до заполнения

Всего пропусков: 0

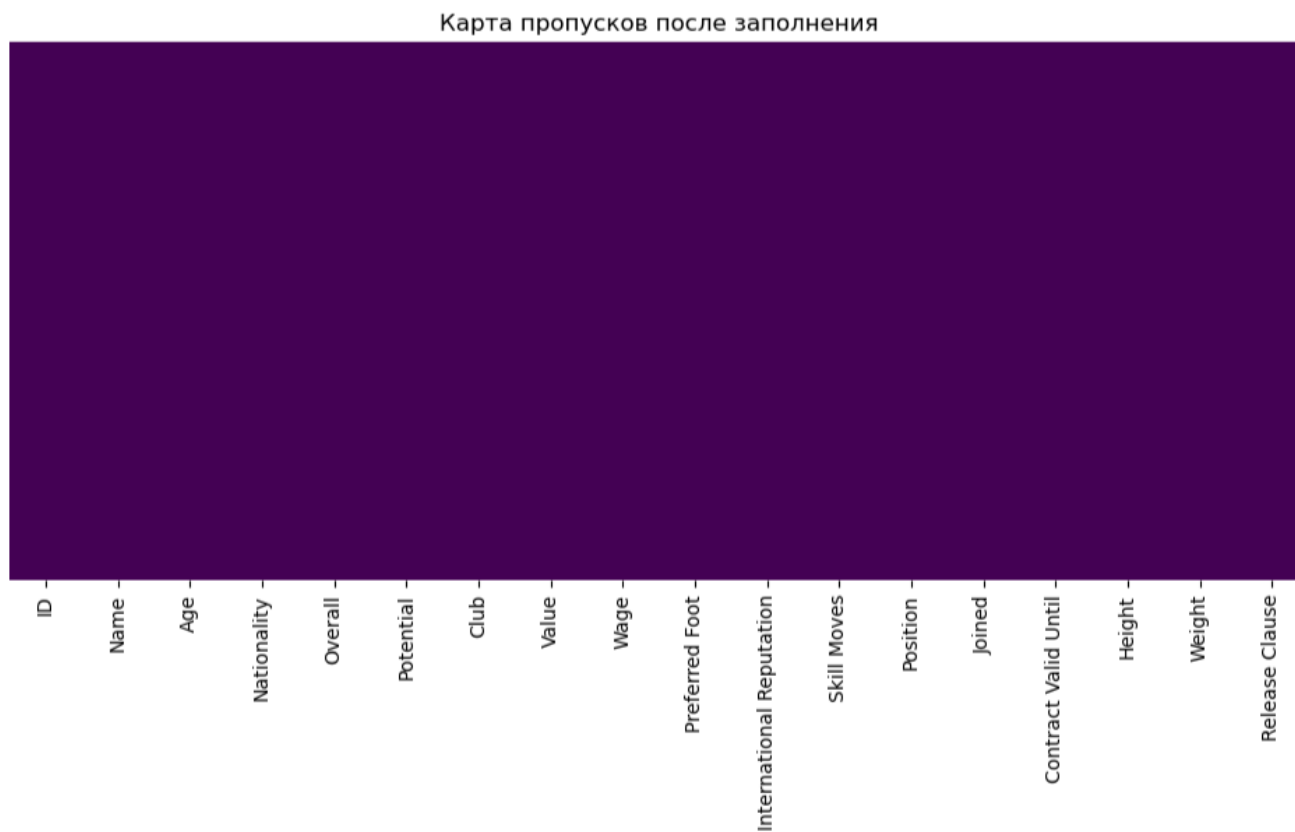


Рис. 3 – Карта пропусков после заполнения

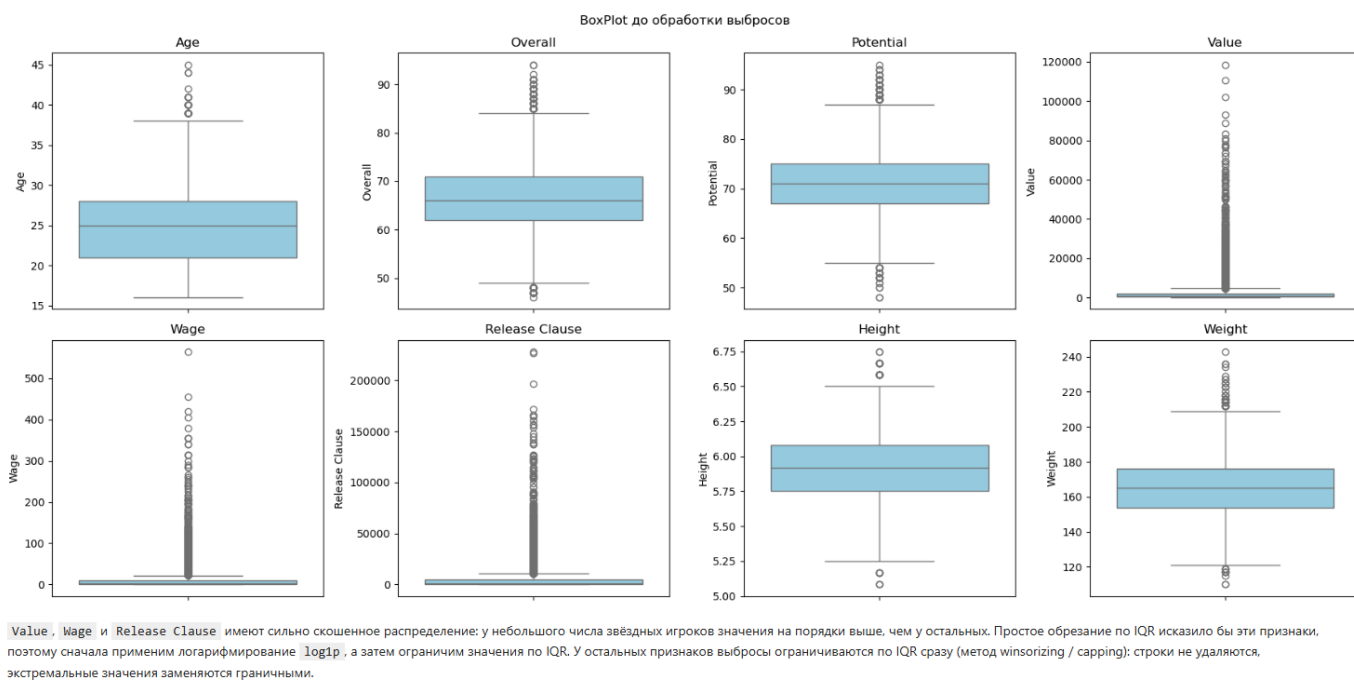


Рис. 4 – BoxPlot числовых признаков до обработки выбросов

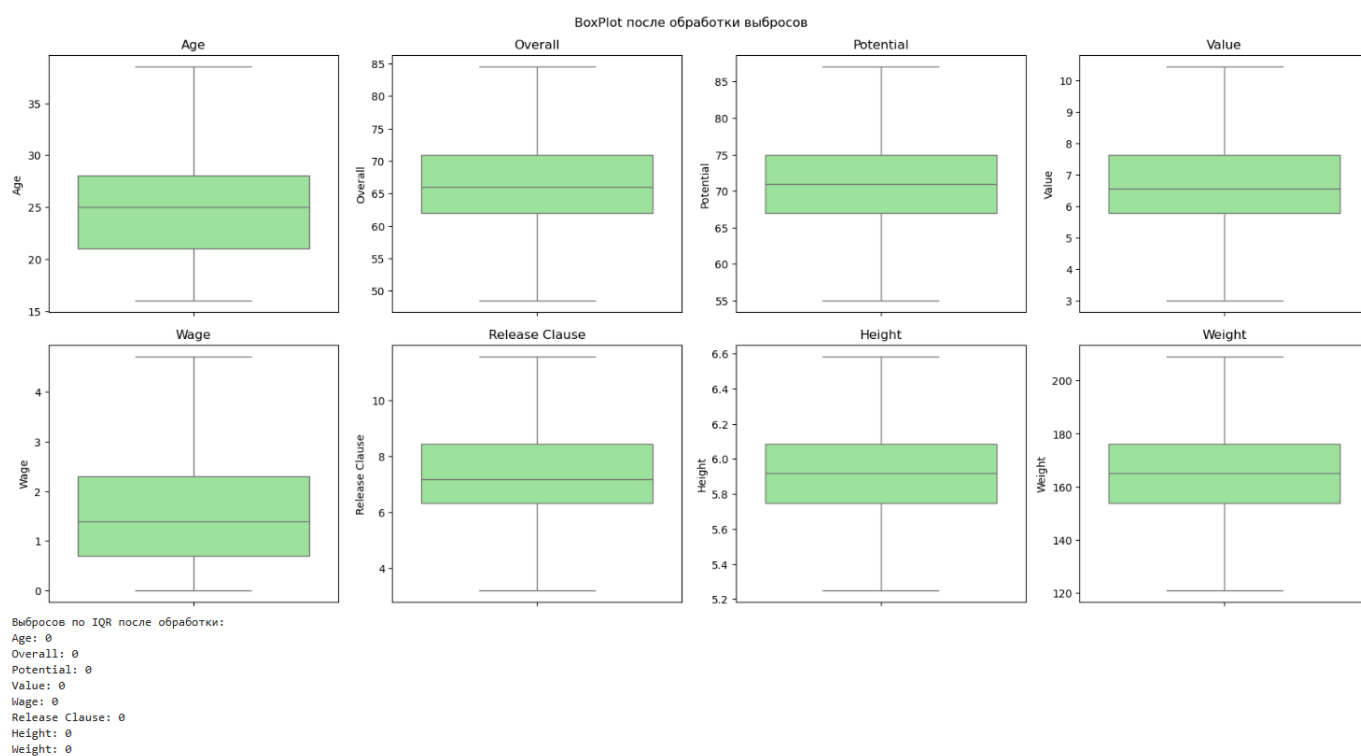


Рис. 5 – BoxPlot числовых признаков после обработки выбросов

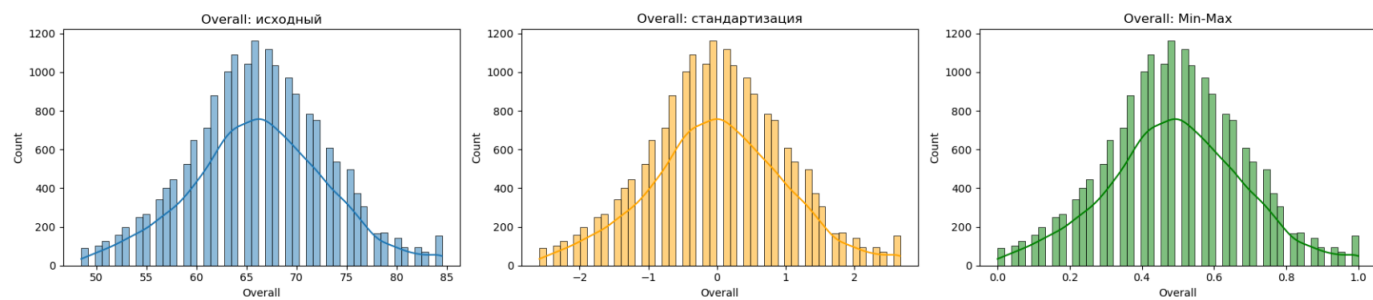


Рис. 6 – Результаты стандартизации и нормализации

6 ВЫВОДЫ ПО ЛАБОРАТОРНОЙ РАБОТЕ

В ходе лабораторной работы выполнен разведочный анализ и подготовка набора данных `fifa_eda` (18 207 записей, 18 столбцов, данные об игроках FIFA 19). Проверка показала отсутствие повторяющихся идентификаторов и полных дубликатов строк.

С помощью `df.info()` и тепловой карты пропусков выявлены пропуски в столбцах `Club`, `Value`, `International Reputation`, `Skill Moves` и `Contract Valid Until`. Пропуски заполнены с учётом смысла признаков: `Club` — значением «Free Agent», `Contract Valid Until` — медианой года, `Value` — медианой по группам `Overall`, порядковые признаки — модой. Повторная тепловая карта подтвердила отсутствие пропусков.

Выбросы обнаружены по диаграммам `BoxPlot` и правилу `IQR`. Денежные признаки `Value`, `Wage` и `Release Clause` предварительно логарифмированы (`log1p`), затем значения всех числовых признаков ограничены границами `IQR`. Повторные `BoxPlot` и подсчёт выбросов подтвердили их устранение.

Сгенерированы новые признаки `Potential_Gap`, `Years_In_Club`, `Contract_Years`, `Age_Group`. Категориальные признаки закодированы бинаризацией (`Is_Free_Agent`, `Is_Right_Foot`, `Is_Young`, `Is_Star`) и ONE-кодированием (группы позиций, возрастные группы, национальность с выделением десяти наиболее частых значений).

Стандартизация привела непрерывные и порядковые признаки к среднему, равному 0, и стандартному отклонению, равному 1; нормализация `Min-Max` — к диапазону от 0 до 1. Бинарные и ONE-столбцы не масштабировались. Подготовленные данные пригодны для дальнейшего построения моделей машинного обучения.